

Excel “Place in Cell” (Rich Value) Images: Format Deep-Dive + Diagnosis of the Off-by-One

xlcellimage

TL;DR

- **The off-by-one is almost certainly at ONE of exactly two hops, both classic 1-based-vs-0-based traps.** Either (a) the code reads the cell's `vm=` attribute and uses it directly as a 0-based list index (`valueMetadata[vm]`) when `vm` is **1-based** (Microsoft's [MS-OI29500] explicitly overrides the ISO spec: “Office specifies that @vm is a one-based index”), `microsoft` or (b) it decrements `vm` correctly but then treats the `rvb i= / LocalImageIdentifier value` — which are genuinely **0-based** — as if they were 1-based, or subtracts 1 somewhere it shouldn't. Getting “the image one before the correct one” is the signature of exactly one hop being off by -1.
- **The single most likely culprit is the `vm - valueMetadata <bk>` hop.** `vm="1"` means the **first** `<bk>`. If the code does `metadata_blocks[vm]` (Python 0-based) instead of `metadata_blocks[vm-1]`, and there are ≥ 2 rich values, you systematically land on the block for the *next* value while the image chain elsewhere is 0-based — but the more common and exactly-symptom-matching version is the reverse: the code decrements the wrong hop. See Part B.
- **I could not retrieve the actual .py source of dalmartin/xlcellimage** (the repo exists and is public — README, pyproject.toml, and directory layout confirmed — but every raw/CDN/PyPI fetch was blocked by tooling). So Part B is a targeted diagnosis against the confirmed API surface plus a drop-in correct reference implementation you can diff against. Part A is complete and fully sourced.

Key Findings

1. **The chain of indirection has SIX hops, and the base (0 vs 1) alternates.** Confirmed against [MS-XLSX], [MS-OI29500], and independent reverse-engineering (Microsoft Fabric community thread by ZhangKun/ronrsnfd; the `excelize`, `xlsx-template`, and `naclomi/xlsx-rich-image-scraper` implementations):
 - `c/@vm` → `valueMetadata/bk` list: **1-based** (Office override; ISO says 0-based — *this disagreement is the #1 bug source*).
 - `bk/rc/@t` → `metadataTypes/metadataType` list: **1-based**.
 - `bk/rc/@v` → `futureMetadata/bk` list: **0-based**.
 - `xlrd:rvb/@i` → `rdrichvalue.xml rv` list: **0-based**.
 - `rv/@s` → `rdrichvaluestructure.xml s` (structure) list: **0-based**.
 - `LocalImageIdentifier value` → `richValueRel.xml rel` list: **0-based** (position in document order, NOT the rld number).
 - `rel/@r (rld)` → `richValueRel.xml.rels` lookup by **Id string**, giving `../media/imageN.png`.
2. **“Image immediately before the correct one” ⇒ one hop that is 1-based was indexed as 0-based (or a spurious -1).** Every hop is a candidate, but the two hops where an implementer is most likely to get the base wrong are the `vm` hop (spec says 1-based but the raw XML also literally contains `vm="1"` for the first value, which looks 0-based-plus-one and invites `list[vm]`) and the final `LocalImageIdentifier` → `rel` hop.
3. **xlcellimage is pure-stdlib** (`xml.etree.ElementTree` or `xml.dom`, no `lxml`, no `openpyxl`). Confirmed from README (“With no dependencies (pure stdlib)”) and `pyproject.toml` (no runtime deps; dev-only `pytest`, `ruff`). This matters: `ElementTree` preserves document order and does NOT include comment/whitespace nodes as element children (unlike some `lxml` usage), so the “whitespace node” failure mode is unlikely here — pushing the diagnosis firmly toward a numeric base error rather than a node-iteration error.

Details

A.1 — The package parts involved

For a workbook with in-cell images you will find (floating/drawing parts are deliberately excluded):

- `xl/worksheets/sheetN.xml` — the cell carrying the reference. A “Place in Cell” image cell looks like:

```
<c r="A1" s="1" t="e" vm="1"><v>#VALUE!</v></c>
```

`t="e"` + fallback `#VALUE!` is the value old Excel shows; the real payload is reached via `vm` (value-metadata index). `cm=` (cell-metadata index) may also appear; for images the reference you want is `vm`. `s` is the style index (unrelated).

- `xl/metadata.xml` — `<metadataTypes>`, one or more `<futureMetadata name="XLRICHVALUE">` (and possibly `name="XLDAPR"` for dynamic arrays), and

xl/richData/_rels/richValueRel.xml.rels:

```
<Relationships xmlns="...">
  <Relationship Id="rId1" Type=".../image" Target="../media/image1.png"/>
  <Relationship Id="rId2" Type=".../image" Target="../media/image2.png"/>
</Relationships>
```

Resolution for `A1 (vm="1")`:

- `vm=1` → **valueMetadata block**. 1-BASED. Block index = `vm - 1 = 0` → first `<bk>`. **[Bug hotspot #1]**
- `rc/@t=1` → **metadata type**. 1-BASED. Type `t-1 = 0` → `metadataTypes[0] = XLRICHVALUE`. **You MUST verify the name is `XLRICHVALUE` before trusting `v` (a block could be `XLDAPR`).**
- `rc/@v=0` → **futureMetadata (name="XLRICHVALUE") block**. 0-BASED. `futureMetadata/bk[0]`.
- `xlrd:rvb/@i=0` → **richvalue.xml rv**. 0-BASED. `rvData/rv[0]`.
- `rv/@s=0` → **structure**. 0-BASED. `rvStructures/s[0]`. Find the position of the key `_rvRel:LocalImageIdentifier` among that structure's `<k>` list → position 0. The corresponding `<v>` in the `rv` is `rv/v[0] = 0`. That value **is the LocalImageIdentifier**. 0-BASED index into `richValueRel`.
- LocalImageIdentifier=0** → **richValueRel.xml rel list position**. 0-BASED. `rel[0] → r="rId1"`. **Use LIST POSITION, not the rId number. [Bug hotspot #2]**
- `rId1` → **.rels lookup by id**. String match → `Target="../media/image1.png"` → **unzip** `xl/media/image1.png`.

A.3 — Common pitfalls (ranked by how often they bite)

- 1-based vm indexed as 0-based** (or vice-versa). Signature: off-by-one, and specifically “image before” if the code does `list[vm]` on a 1-based value but everything downstream is 0-based... actually the more reliable “one before” signature is a spurious decrement on an already-0-based hop, OR failing to decrement `vm`. Note the [MS-OI29500] override: standard says `@vm / @cm` are 0-based; **Office writes them 1-based**. microsoft Any implementer who read only ISO/IEC 29500 gets this wrong.
- Using the rId numeric suffix instead of `<rel>` document-order position**. The `LocalImageIdentifier` indexes the *ordered list of `<rel>` elements*, not `rId{n}`. These coincide only when rels are written `rId1, rId2, ...` in order. If Excel reuses/reorders/deletes, `rId10` can precede `rId2`, and the numeric suffix diverges from position. This produces mis-ordered (not merely off-by-one) results.
- Sorting rIds lexicographically** (`"rId10" < "rId2"`) — scrambles everything once you exceed 9 images.
- Assuming `media/imageN.png` numbering == rich-value order**. Media is shared across features; a floating drawing or workbook logo also lives in `xl/media/`, so listing `xl/media/` and indexing by position is wrong.
- Non-image rich values occupying `rv` slots**. Stocks/Geography/linked-data-type entities and Python-result rich values are also `rv` entries. The Nth `rv` is not the Nth image. Always resolve via structure `t="_localImage"` and the `_rvRel:LocalImageIdentifier` key — never by counting images.
- Picking the wrong `<rc>`**. A `<bk>` may contain multiple `<rc>` records (e.g., both `XLDAPR` dynamic-array and `XLRICHVALUE`). You must select the `rc` whose type resolves to `XLRICHVALUE`.
- s (structure) index assumed constant = 0**. With multiple structures, the `LocalImageIdentifier` may be at a different key position per structure. Look it up per-`rv`.
- Deleted/orphaned rich values leaving gaps**, and images referenced by multiple cells (same `rIdN` for several cells) — fine if you resolve per-cell, broken if you zip cells-to-images positionally.
- XML namespaces**: `rvb` is in the 2017/richdata namespace; `richValueRels / rel` in 2022/richvaluerel; `r:Id` on `rel` is in the standard relationships namespace. Namespace-blind `find()` calls silently return nothing.
- .xism VS .xlsx**: identical rich-data layout; only `[Content_Types].xml / VBA` differ. Streaming vs full parse doesn't change the algorithm, but a streaming reader that discards element order will corrupt the list-position hops.

Where sources disagree on base: ISO/IEC 29500 vs Microsoft on `@vm / @cm` (0-based vs 1-based — resolved in Office's favor: **1-based**). Everything inside `richData (rvb/@i, rv/@s, the "r" / "a" / "spb" reference types, and LocalImageIdentifier)` is uniformly documented by [MS-XLSX] as **0-based**. The `rc/@t` type index and the `vm` block index are the 1-based ones.

Part B — Code review of `dalmartin/xlcellimage`

Access status (stated plainly): The repository is public and exists — I confirmed the README, the `pyproject.toml` (verbatim: `version=0.1.0, requires-python>=3.10, no runtime dependencies, dev-only pytest / ruff, build backend hatchling`), and the layout: `src/xlcellimage/` (package, exports `ImageLoader` via `__init__.py`), `tests/`, `.github/workflows/`, `LICENSE`, `README.md`. **I was unable to fetch the actual .py source** (raw.githubusercontent.com, jsDelivr, and PyPI file fetches were all blocked by the fetch tool's allowlist, and GitHub does not expose these small files to search). I will not invent the code. What follows is a precise diagnosis constrained by the confirmed API and the format facts above, plus a corrected

reference implementation.

Confirmed API surface (from README):

- `ImageLoader(path_to_workbook: str)` — “runs it through a parser to **map cells with images to image filepaths**” (eager, full-parse, builds a dict at construction).
- `hasImage(sheet, cell) -> bool`
- `getImage(sheet, cell) -> bytes`

Architectural read: A pure-stdlib, eager, whole-workbook parse into a `{(sheet, cell): image_path}` map is a reasonable design for small workbooks. It is fine. Two architectural cautions: (1) an eager full parse defeats the “efficient” claim on large workbooks and holds the zip open assumptions; (2) mapping to *filepaths* internally then re-opening is fine, but the cell→image mapping must be built by walking the per-cell `vm` chain, **not** by zipping “cells that have `vm`” against “images in `xl/media`” or against “`rv` entries in order.” If the code builds the map by any positional zip, it is architecturally wrong and will fail on every workbook that has a non-image rich value, a floating image in `xl/media`, or a deleted value — independent of the off-by-one.

The off-by-one diagnosis

Because you report “**the image one before the correct one,**” the defect is a single `-1` error on exactly one hop, and it is stable across cells (not random) — which rules out lexicographic `rId` sorting (that scrambles, not shifts) and rules out whitespace-node inclusion (`ElementTree`, being `stdlib`, doesn’t add those). That leaves a numeric base error. Ranked:

1. Most likely: the `vm - valueMetadata <bk>` hop is decremented when it should not be, OR a downstream 0-based hop is decremented.

Concretely, the symptom “image[n-1]” is produced when the code takes an index that is *already correct/0-based* and subtracts 1 from it. The prime suspect is code that “helpfully” applies `vm - 1` **and also** does not need to (because it already located the block by iterating), or applies `-1` to `rvb/@i` / to the `LocalImageIdentifier` value. If the code does something like:

```
rel_index = local_image_id - 1          # WRONG: LocalImageIdentifier is already 0-based
rel = rels_list[rel_index]
```

you get exactly “one image before.” **Corrected:** `rel = rels_list[local_image_id]` (no decrement).

2. Equally plausible mirror-image bug: the code correctly leaves the 0-based hops alone but forgets that `vm` is 1-based, i.e. `block = value_meta[vm]` should be `value_meta[vm - 1]`. If the rest of the chain is 0-based and correct, `value_meta[vm]` lands on the block for the **next** rich value → you’d get “the image after,” not before. So if your symptom is strictly “before,” bug #1 (a spurious `-1` on an already-0-based hop, most likely the final `LocalImageIdentifier` → `rel` hop) is the better fit. **The final hop is where the `-1` most often creeps in**, because developers see `rId1` as “the first” and mentally map `LocalImageIdentifier=0 / 1 / ...` to `rId1 / rId2 / ...` by writing `rels[id-1]` — which is right only if the value were 1-based, but it is 0-based, so it fetches one too early.

State the corrected logic bluntly:

- `valueMetadata` block index = `vm - 1` (because `vm` is 1-based).
- `metadataType` index = `t - 1` (1-based); confirm `name == "XLRICHVALUE"`.
- `futureMetadata` block index = `v` (0-based, no decrement).
- `rv` index = `i` from `rvb` (0-based, no decrement).
- `rel` list position = `LocalImageIdentifier` (0-based, no decrement) — and resolve `rel/@r` to a Target **by Id string**, never by number.

If the library currently produces “one before” uniformly, search the source for any `- 1` applied to: the `rvb i` value, the `rv/v` `LocalImageIdentifier` value, or the `rel`-list index. Delete that decrement. If instead it uses `int(rId[3:])` (numeric suffix) to index `rels`, replace with document-order position of the `<rel>` element.

Other ways the code is (probably) wrong beyond the off-by-one

- **Non-image rich values / positional zipping.** If the map is built by pairing `vm`-bearing cells with images positionally, it breaks on Stocks/Geography/Python rich values and on workbooks whose `xl/media` also holds floating images or a logo. Fix: resolve each cell’s own chain and only accept structures with `t=="_localImage"`.
- **cm present as well as vm.** If a cell has both, the code must use `vm` for the image; using `cm` (cell metadata) fetches the wrong block or none.
- **rId ordering assumptions.** Any reliance on `rId{n}` numeric order or lexicographic sort fails for ≥ 10 images or reordered `rels`. Use `<rel>` element document order + `.rels` Id-string lookup.
- **Namespace-naive find / findall.** `rvb`, `richValueRels`, and `r:Id` live in three different namespaces; `stdlib` `ElementTree` requires the `{uri}local` form or a namespace map. A missing namespace silently yields `None / []` and can masquerade as “no image.”
- **Sheet-name → sheet-file mapping.** README’s `hasImage("sheet1", "A1")` implies mapping a display name to `xl/worksheets/sheetN.xml`. If the code assumes `sheet` name `"sheetN" == file sheetN.xml`, it’s wrong: the mapping must go through `workbook.xml <sheet name r:Id> + workbook.xml.rels`. Sheet display names $\neq$ part names, and sheet order $\neq$ file-number order.

- **Multiple <rc> per <bk> / XLDAPR blocks.** Must select the `XLRIICHVALUE` `rc`, not the first `rc`.
- **.xslm and alternate image extensions.** Ensure `.jpeg/.gif/.webp` targets are handled (don't hardcode `.png`), and that `.xslm` is accepted.
- **Sparse/absent metadata.** If `xl/metadata.xml` or `xl/richData/*` is missing, `getImage` should cleanly report "no image," not throw an opaque `KeyError` / `IndexError`.
- **Deleted values / gaps and shared images.** Resolve strictly per-cell; never assume contiguous or unique image usage.

Corrected reference implementation (pure stdlib — diff against your code)

```
import re, zipfile
import xml.etree.ElementTree as ET

MAIN = "http://schemas.openxmlformats.org/spreadsheetml/2006/main"
RICHDATA = "http://schemas.microsoft.com/office/spreadsheetml/2017/richdata"
RVREL = "http://schemas.microsoft.com/office/spreadsheetml/2022/richvaluerel"
RELS = "http://schemas.openxmlformats.org/package/2006/relationships"
ODOC_REL = "http://schemas.openxmlformats.org/officeDocument/2006/relationships"

def _q(ns, tag): return f"{{{ns}}}{tag}"

class ImageLoader:
    def __init__(self, path):
        self.path = path
        self.z = zipfile.ZipFile(path)
        self._sheet_part = self._map_sheets()           # display name -> worksheets part
        self._value_meta = self._parse_metadata()       # list of (type_name, future_v)
        self._future = self._parse_future()             # list of rvb i (0-based)
        self._rv = self._parse_richvalue()              # list of (structure_s, [v...])
        self._structs = self._parse_structures()        # list of [key_name...]
        self._rel_order = self._parse_rel_order()       # list position -> rId
        self._rel_target = self._parse_rel_targets()     # rId -> media part path

    # --- sheet name -> part (via workbook.xml + workbook.xml.rels) ---
    def _map_sheets(self):
        wb = ET.fromstring(self.z.read("xl/workbook.xml"))
        rels = ET.fromstring(self.z.read("xl/_rels/workbook.xml.rels"))
        rid2tgt = {r.get("Id"): r.get("Target") for r in rels}
        out = {}
        for s in wb.iter(_q(MAIN, "sheet")):
            rid = s.get(_q(ODOC_REL, "id"))
            tgt = rid2tgt[rid]
            if not tgt.startswith("xl/"): tgt = "xl/" + tgt.lstrip("/")
            out[s.get("name")] = tgt
        return out

    def _parse_metadata(self):
        root = ET.fromstring(self.z.read("xl/metadata.xml"))
        types = [t.get("name") for t in root.iter(_q(MAIN, "metadataType"))] # 1-based via t
        blocks = []
        vm = root.find(_q(MAIN, "valueMetadata"))
        for bk in (vm or []):
            rc = None
            for cand in bk.findall(_q(MAIN, "rc")):
                tname = types[int(cand.get("t")) - 1] # @t is 1-based
                if tname == "XLRIICHVALUE":
                    rc = cand; break
            if rc is None:
                blocks.append(None)
            else:
                blocks.append(("XLRIICHVALUE", int(rc.get("v")))) # @v is 0-based
        return blocks

    def _parse_future(self):
        root = ET.fromstring(self.z.read("xl/metadata.xml"))
        for fm in root.iter(_q(MAIN, "futureMetadata")):
            if fm.get("name") == "XLRIICHVALUE":
                return [int(rvb.get("i")) for rvb in fm.iter(_q(RICHDATA, "rvb"))] # 0-based
        return []

    def _parse_richvalue(self):
        root = ET.fromstring(self.z.read("xl/richData/rdrichvalue.xml"))
        out = []
        for rv in root.iter(_q(RICHDATA, "rv")):
```

```

        vals = [v.text for v in rv.findall(_q(RICHDATA, "v"))]
        out.append((int(rv.get("s")), vals))          # s is 0-based
    return out

def _parse_structures(self):
    root = ET.fromstring(self.z.read("xl/richData/rdrichvaluestructure.xml"))
    out = []
    for s in root.iter(_q(RICHDATA, "s")):
        out.append([k.get("n") for k in s.findall(_q(RICHDATA, "k"))])
    return out

def _parse_rel_order(self):
    root = ET.fromstring(self.z.read("xl/richData/richValueRel.xml"))
    # DOCUMENT ORDER of <rel>; index by position, NOT by rId number
    return [rel.get(_q(ODOC_REL, "id")) for rel in root.findall(_q(RVREL, "rel"))]

def _parse_rel_targets(self):
    root = ET.fromstring(self.z.read("xl/richData/_rels/richValueRel.xml.rels"))
    out = {}
    for r in root:
        tgt = r.get("Target")
        # targets are relative to xl/richData/
        path = "xl/richData/" + tgt if not tgt.startswith("/") else tgt.lstrip("/")
        # normalise ../media -> xl/media
        path = re.sub(r"xl/richData/\\.\\./", "xl/", path)
        out[r.get("Id")] = path
    return out

def _cell_vm(self, sheet, cell):
    part = self._sheet_part[sheet]
    root = ET.fromstring(self.z.read(part))
    for c in root.iter(_q(MAIN, "c")):
        if c.get("r") == cell:
            return c.get("vm")    # string or None
    return None

def _resolve(self, sheet, cell):
    vm = self._cell_vm(sheet, cell)
    if vm is None: return None
    block = self._value_meta[int(vm) - 1]          # vm is 1-BASED <-- key fix
    if block is None or block[0] != "XLRICHVALUE": return None
    fut_v = block[1]                               # 0-based
    rv_index = self._future[fut_v]                 # rvb i, 0-based
    struct_s, vals = self._rv[rv_index]            # 0-based
    keys = self._structs[struct_s]
    try:
        key_pos = keys.index("_rvRel:LocalImageIdentifier")
    except ValueError:
        return None                                # not a local image
    local_image_id = int(vals[key_pos])             # 0-BASED, do NOT decrement
    rid = self._rel_order[local_image_id]          # by list position
    return self._rel_target[rid]

def hasImage(self, sheet, cell):
    return self._resolve(sheet, cell) is not None

def getImage(self, sheet, cell):
    path = self._resolve(sheet, cell)
    if path is None:
        raise KeyError(f"No in-cell image at {sheet}!{cell}")
    return self.z.read(path)

```

The two lines that fix a “one image before” bug are annotated: `int(vm) - 1` (vm is 1-based) and `int(vals[key_pos])` with **no** `- 1` (LocalImageIdentifier is 0-based). If your current code has a `- 1` on the LocalImageIdentifier or on `rvb i`, that is your bug; remove it. If it lacks the `- 1` on `vm`, add it.

Recommendations

1. **Immediately:** grep the source for `- 1`, `-1`, `int(rId, rId[ , .sort( , sorted( , [1:]`, and any indexing of a rels/media list. The off-by-one is one of these. Apply the base rules from A.2: decrement only `vm` and `rc/@t`; never decrement `rvb/@i`, `rv/@s`, or `LocalImageIdentifier`.
2. **Replace any rId-number logic with <rel> document-order position** and a string-keyed `.rels` lookup. Benchmark that fixes correctness for ≥ 10 images.

3. **Add regression fixtures** that specifically break naive code: (a) a workbook with a Stocks/Geography rich value AND an image (non-image `rv` slot), (b) a workbook with a floating drawing image in `xl/media` plus in-cell images (media-order $\neq$ rv-order), (c) ≥ 10 in-cell images (rld10 ordering), (d) a deleted-then-re-added image (rld gap), (e) same image in two cells (shared rld), (f) an `.xism`, (g) a `.jpeg` in-cell image. If all pass, the indexing is correct.
4. **Guard the missing-part case:** if `xl/metadata.xml` or `xl/richData/*` is absent, `hasImage` returns `False` and `getImage` raises a clear error — not `IndexError`.
5. **Threshold that changes the recommendation:** if you must support very large workbooks, switch from eager full-map construction to lazy per-cell resolution (parse metadata/richData once, resolve the cell's `vm` chain on demand). Keep eager only while workbooks are small.

Caveats

- **Part B is a diagnosis, not a line-cited review**, because the `.py` files were not retrievable (repo confirmed public; fetch blocked by tooling, not by the repo). The corrected algorithm and base annotations are authoritative ([MS-XLSX], [MS-OI29500]); the exact offending line in your code must be confirmed by grepping per the Recommendations. My confidence that the bug is a single spurious `-1` /base error (most likely on the final `LocalImageIdentifier` $\rightarrow$ rel hop, secondarily the `vm` hop) is high given the “one image before” symptom and the pure-stdlib design (which excludes lxml whitespace-node and reordering causes).
- The `vm / cm` 1-based behavior is a **Microsoft Office override of ISO/IEC 29500** (which says 0-based); this is documented in [MS-OI29500] §2fd4e47f. Code written to the ISO text alone will be wrong. All richData indices are 0-based per [MS-XLSX] CT_RichValue and the Local Image Type KVP (“`_rvRel:LocalImageIdentifier` ... a zero-based index ... MUST reference a CT_RichValueRelRelationship”).
- Reverse-engineering corroboration comes from the Microsoft Fabric community thread (ZhangKun’s step list, explicitly noting “some indexes start from 0 ... and some start from 1 (such as the vm attribute value)”), the `qax-os/excelize` struct definitions, the `optilude/xlsx-template` issue #186 (worked XML example), and `naclomi/xlsx-rich-image-scraper` (an existing Python implementation of this exact chain you can also diff against). Microsoft’s own prose docs are thin here, so these community sources carry real weight.